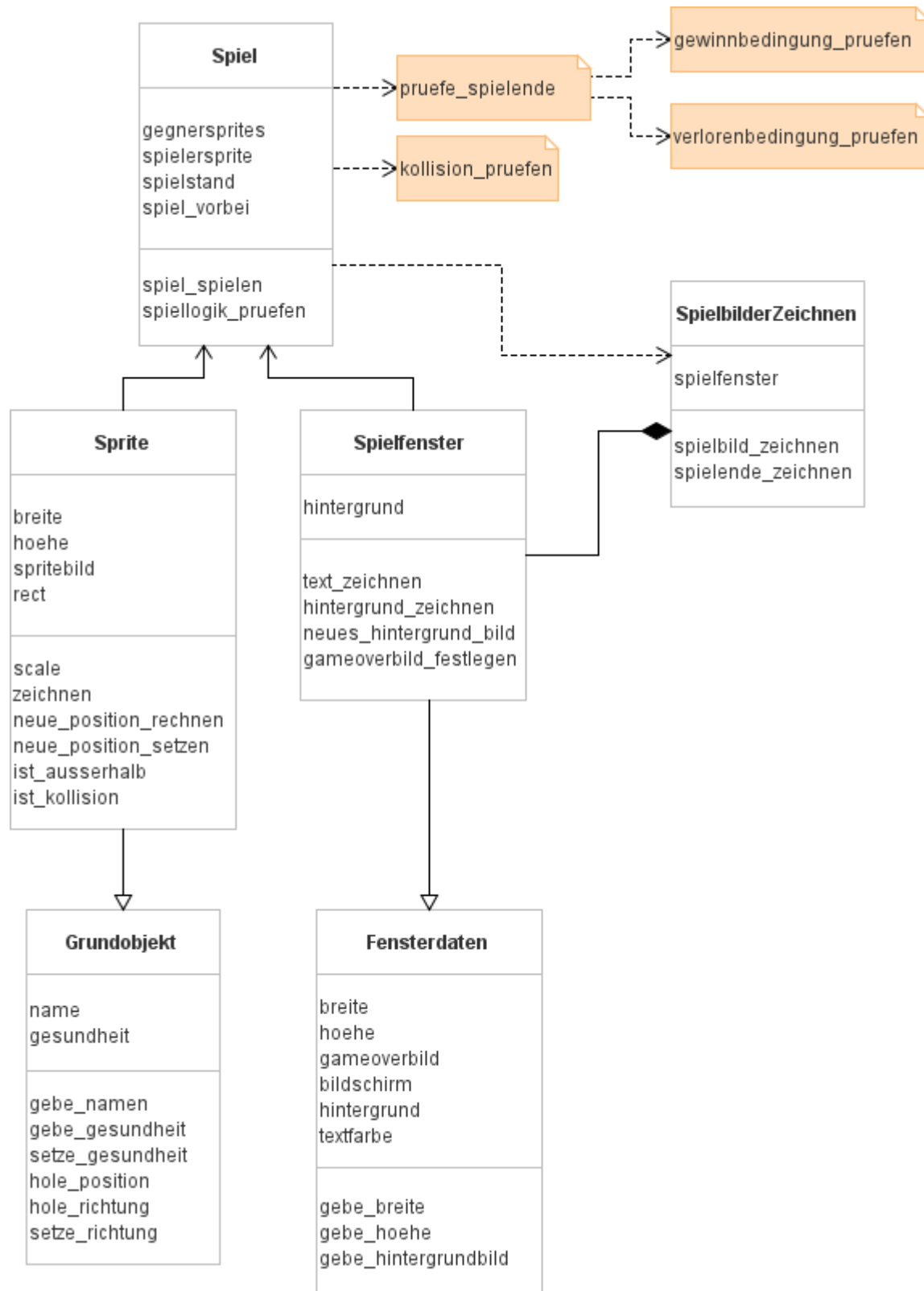


Übersicht über das Spiel



Übersicht über die Module

spiel.py

Class Spiel

- 1. Initialisierung:** Die Funktion `spiel_spielen` nimmt zwei Parameter: `gegnersprites` und `spielersprite`. Diese repräsentieren die Sprites des Gegners und des Spielers im Spiel. Zunächst werden einige Variablen initialisiert:
 - `spiel_vorbei` wird auf `False` gesetzt. Diese Variable wird verwendet, um zu überprüfen, ob das Spiel beendet ist.
 - `spielstand` wird auf `0` gesetzt. Diese Variable hält den aktuellen Spielstand.
 - Ein `clock`-Objekt wird erstellt, um die Framerate des Spiels zu kontrollieren.
 - Der Mauszeiger wird mit `pygame.mouse.set_visible(False)` ausgeblendet.
- 2. Hauptschleife:** Die Hauptschleife des Spiels beginnt mit `while not spiel_vorbei:`. Diese Schleife läuft so lange, bis `spiel_vorbei` auf `True` gesetzt wird.
 - **Ereignisschleife:** Innerhalb der Hauptschleife gibt es eine weitere Schleife, die alle Pygame-Ereignisse durchläuft. Wenn das Ereignis `pygame.QUIT` (normalerweise ausgelöst durch das Schließen des Fensters) auftritt, wird `spiel_vorbei` auf `True` gesetzt und die Hauptschleife endet.
 - **Spiellogik prüfen:** Die Funktion `spiellogik_pruefen` wird aufgerufen, um die Spiellogik zu überprüfen und den Spielstand und den Spielstatus zu aktualisieren.
 - **Spielbild zeichnen:** Die Funktion `spielbild_zeichnen` wird aufgerufen, um das Spielfeld zu zeichnen.
 - **Framerate kontrollieren:** `clock.tick(60)` wird aufgerufen, um sicherzustellen, dass das Spiel mit 60 Bildern pro Sekunde läuft.
- 3. Spielende:** Wenn die Hauptschleife endet (d.h., das Spiel ist vorbei), wird die Funktion `spielende_zeichnen` aufgerufen, um das Endbildschirm zu zeichnen. Schließlich wird `pygame.quit()` aufgerufen, um Pygame zu beenden und alle Ressourcen freizugeben.

spiellogik_pruefen()

1. **Kollision prüfen:** Die Funktion `kollision_pruefen` wird aufgerufen, um zu überprüfen, ob eine Kollision zwischen dem Spieler und den Gegnern stattgefunden hat. Diese Funktion nimmt `spielersprite` und `gegnersprites` als Parameter und gibt den aktualisierten Spielstand zurück, der dann zum aktuellen `spielstand` addiert wird.
2. **Spielende prüfen:** Die Funktion `pruefe_spielende` wird aufgerufen, um zu überprüfen, ob das Spiel beendet ist. Diese Funktion nimmt `spielstand` und einen optionalen `parameter` als Parameter und gibt einen booleschen Wert zurück, der angibt, ob das Spiel vorbei ist (`True`) oder nicht (`False`). Dieser Wert wird dann der Variable `spiel_vorbei` zugewiesen.
3. **Bewegung aktualisieren:** Die Funktion `bewegung_aktualisieren` wird aufgerufen, um die Bewegung des Spielers und der Gegner zu aktualisieren. Diese Funktion nimmt `spielersprite`, `gegnersprites` und die aktuelle Position der Maus (`pygame.mouse.get_pos()`) als Parameter.
4. **Rückgabewerte:** Schließlich gibt die Funktion `spiellogik_pruefen` den aktualisierten `spielstand` und den Status `spiel_vorbei` zurück.

hintergrundlogik.py

1. `kollision_pruefen(spielersprite, gegnersprite)`: Diese Funktion prüft, ob es eine Kollision zwischen dem Spieler und dem Gegner gibt. Wenn es mehrere Gegner gibt (d.h., wenn `gegnersprite` eine Liste ist), wird für jeden Gegner geprüft, ob es eine Kollision gibt. Wenn es nur einen Gegner gibt, wird nur für diesen geprüft. Die Funktion gibt die Anzahl der Kollisionen zurück.
2. `pruefe_spielende(spielstand, parameter = None)`: Diese Funktion prüft, ob das Spiel zu Ende ist. Sie nimmt zwei Parameter entgegen, die für die Prüfung benötigt werden. Die Funktion gibt `True` zurück, wenn das Spiel gewonnen ist und nicht verloren wurde, `False` in allen anderen Fällen.
3. `bewegung_aktualisieren(spielersprite, gegnersprite, maus_position)`: Diese Funktion aktualisiert die Bewegung der Gegner und des Spielers. Sie bewegt die Gegner über deren Methode `neue_position_rechnen()` und prüft, ob sie aus dem Spielfeld heraus sind. Sie setzt auch die Position des Spielers auf die aktuelle Mausposition.
4. `gewinnbedingung_pruefen(spielstand)`: Diese Funktion prüft, ob das Spiel gewonnen ist. In dieser Spielvariante wird das Spiel gewonnen, wenn der Spielstand 100 erreicht hat. Sie gibt `True` zurück, wenn das Spiel gewonnen wurde, und `False` sonst.
5. `verlorenbedingung_pruefen(parameter = None)`: Diese Funktion prüft, ob das Spiel verloren ist. Sie nimmt einen Parameter entgegen, der für die Prüfung benötigt wird. Derzeit gibt die Funktion immer `False` zurück, was bedeutet, dass das Spiel im Moment noch nicht verloren werden kann.

grundobjekt.py:

Die Klasse Grundobjekt hält weitere Methoden bereit, die wir noch nicht nutzen.

- `gebe_namen(self)`: Diese Methode gibt den Namen des Sprites zurück.
- `gebe_gesundheit(self)`: Diese Methode gibt die Gesundheit des Sprites zurück.
- `setze_gesundheit(self, neue_energie)`: Diese Methode setzt die Gesundheit des Sprites auf den übergebenen Wert.
- `hole_richtung(self)`: Diese Methode gibt die aktuelle Bewegungsrichtung des Sprites zurück.
- `setze_richtung(self, neue_richtung)`: Diese Methode setzt die Bewegungsrichtung des Sprites auf die übergebenen Koordinaten.

spielobjekte.py

2. **Bilder für die Sprites:** Zuerst werden die Bilder für die Sprites geladen und skaliert. Ein Sprite ist ein 2D-Bild, das in einem Videospiel verwendet wird. In diesem Fall gibt es drei Sprites: einen Roboter und zwei Arten von Asteroiden.
3. **Die Sprite-Klasse:** Diese Klasse definiert ein Sprite-Objekt. Jedes Sprite hat eine Position, eine Größe, eine Gesundheit und eine Bewegungsrichtung. Die Klasse hat auch mehrere Methoden:
 - `__init__(self, x, y, breite, hoehe, richtung, bildpfad, name, gesundheit)`: Dies ist der Konstruktor der Klasse. Er initialisiert die Attribute des Sprites, wie die Position (x, y), die Größe (breite, hoehe), die Bewegungsrichtung (richtung), das Bild (bildpfad), den Namen (name) und die Gesundheit (gesundheit).
 - `scale(self, breite, hoehe)`: Diese Methode skaliert das Bild des Sprites auf die gegebene Breite und Höhe.
 - `zeichnen(self, fensterobjekt)`: Diese Methode zeichnet das Sprite auf das Spielfenster.
 - `neue_position_rechnen(self)`: Diese Methode berechnet die neue Position des Sprites, wenn dies ein NPC ist.
 - `neue_position_setzen(self, x, y)`: Diese Methode setzt die Position des Sprites auf die übergebenen Koordinaten.
 - `hole_position(self)`: Diese Methode gibt die aktuelle Position des Sprites zurück.
 - `ist_ausserhalb(self)`: Diese Methode prüft, ob das Sprite aus dem Spielfeld heraus ist und setzt es gegebenenfalls wieder hinein.
 - `ist_kollision(self, spieler)`: Diese Methode prüft, ob es eine Kollision mit einem anderen Sprite gibt.
4. **Erzeugung der Sprites:** Am Ende des Skripts werden die Sprites für den Spieler und die Gegner erzeugt. Der Spieler ist ein Roboter und die Gegner sind Asteroiden. Jeder Gegner wird mit einer bestimmten Position, Größe, Bewegungsrichtung, Bildpfad, Namen und Gesundheit erstellt. Der Spieler wird ebenfalls mit einer bestimmten Position, Größe, Bewegungsrichtung, Bildpfad, Namen und Gesundheit erstellt.